

ABSTRACT

The analysis of a DC machine (DC motor) is presented starting from a first-order model of said system.

This work presents the modelling, simulation, and analysis of a DC motor speed control system using a Proportional-Integral-Derivative (PID) controller in MATLAB Simulink. The dynamic behaviour of the motor is derived from fundamental electrical and mechanical equations, leading to a transfer function representation. A simplified first-order approximation is used for simulation purposes. The system is analyzed under three configurations: without a controller, with manually implemented PID, and with a tuned PID controller. The results demonstrate significant improvement in transient response, stability, and steady-state accuracy with PID control.

INTRODUCTION

The importance in the design of the control systems has grown during the last years to a great extent, taking a considerable participation in the industry.

Currently, control devices are designed for different elements and processes of companies. Electric machines then play an important role in such systems, and the study of stability and control of these also does so, because the processes often use DC motors to drive their workloads. These engines have quite simple functional and constructive models.

Control systems play a crucial role in modern engineering applications, especially in industrial automation and embedded systems. DC motors are widely used due to their simplicity, reliability, and ease of control.

Among various control techniques, the PID controller remains one of the most effective and widely implemented due to its simplicity and robustness. This work focuses on designing and analyzing a PID-based speed control system for a DC motor using simulation tools.

1. MATHEMATICAL MODELLING OF DC MOTOR

A DC motor converts electrical energy into mechanical energy. Its dynamic behaviour can be described using electrical and mechanical equations.

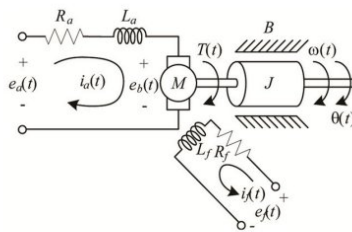


Figure 1: DC motor.

1.1 Electrical Dynamics

Using Kirchhoff's voltage law:

$$V(t) = L \frac{di(t)}{dt} + Ri(t) + e_b(t)$$

Where:

- $V(t)$ = input voltage
- L = armature inductance
- R = armature resistance
- $i(t)$ = armature current
- $e_b(t)$ = back EMF
- The back EMF is proportional to motor speed:

$$e_b(t) = K_b \omega(t)$$

1.2 Mechanical Dynamics

Using Newton's law:

$$J \frac{d\omega(t)}{dt} + b\omega(t) = T(t)$$

Where:

- J = moment of inertia
- b = friction coefficient
- $\omega(t)$ = angular speed
- $T(t)$ = torque

Torque is related to current:

$$T(t) = K_t i(t)$$

1.3 Combined System Model

Taking Laplace transform:

$$G(s) = \frac{\Omega(s)}{V(s)} = \frac{K_t}{(Js + b)(Ls + R) + K_t K_b}$$

1.4 Simplified Model Used

To reduce complexity, the system is approximated as:

$$G(s) = \frac{1}{\tau s + 1}$$

For this project:

$$G(s) = \frac{1}{0.5s + 1}$$

This model represents the relationship between input voltage and output speed.

2. PID CONTROLLER DESIGN

The PID controller is defined as:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

In Laplace form:

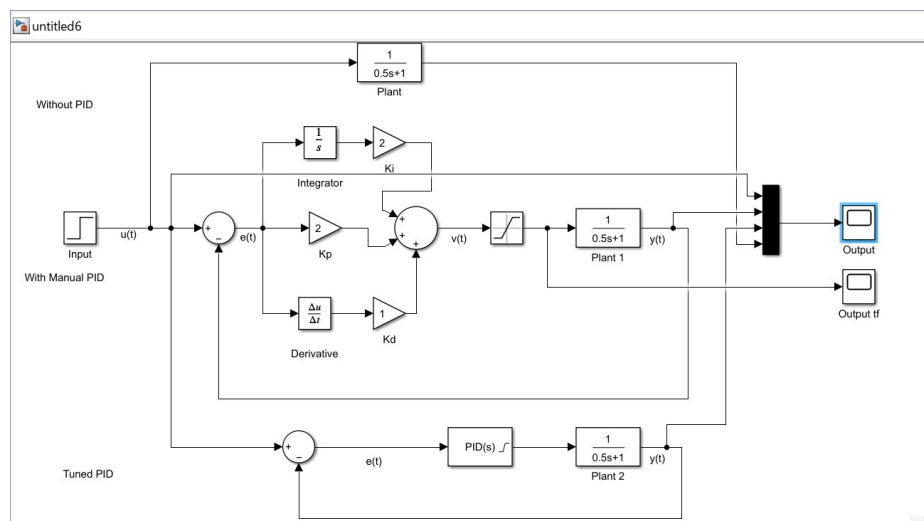
$$C(s) = K_p + \frac{K_i}{s} + K_d s$$

2.1 Closed Loop System

$$T(s) = \frac{G(s)C(s)}{1 + G(s)C(s)}$$

3. PID TUNING AND EFFECT OF PARAMETERS

In this project, the PID controller parameters were tuned manually by observing the system response in Simulink. The tuning process involved adjusting the proportional, integral, and derivative gains to achieve a balance between speed, stability, and accuracy.



PID MODEL

3.1 Effect of Proportional Gain (K_p)

The proportional gain determines how strongly the controller reacts to the current error.

- Increasing K_p reduces rise time and makes the system respond faster.
- However, too high K_p increases overshoot and may cause oscillations.
- A very low K_p results in a slow response.

3.2 Effect of Integral Gain (K_i)

The integral gain accounts for the accumulation of past errors.

- Increasing K_i eliminates steady-state error
- It ensures that the output eventually reaches the desired value
- However, high K_i can increase overshoot and settling time

3.3 Effect of Derivative Gain (K_d)

The derivative gain predicts future error by considering the rate of change.

- Increasing K_d reduces overshoot.
- It improves system stability.
- Excessive K_d can make the system sensitive to noise.

4. PID TUNING APPROACH USED

In this work, PID parameters were tuned manually by the trial-and-error method:

1. K_p was increased to obtain a faster response.
2. K_i was added to eliminate steady-state error.
3. K_d was adjusted to reduce overshoot and improve stability.

Additionally, Simulink's built-in PID block was used to obtain a tuned response for comparison.

5. SIMULATION METHODOLOGY

The system is modelled in MATLAB Simulink using a closed-loop configuration consisting of:

5.1 Main Blocks Used

The model consists of the following blocks:

- **Step Input:** Provides the desired reference speed
- **Sum Block:** Computes error between reference input and output
- **PID Controller Block:** Implements tuned PID control
- **Transfer Function:** Represents the DC motor model
- **Feedback Path:** Returns output to input for closed-loop control
- **Scope:** Displays system response

5.2 Additional Blocks Used

To implement manual PID control and compare different cases, the following blocks are also used:

- **Gain Blocks:** Used to represent proportional, integral, and derivative gains (K_p, K_i, K_d)
- **Integrator Block:** Implements the integral action $\int e(t)dt$
- **Derivative Block:** Implements the derivative action $\frac{de(t)}{dt}$
- **Mux Block:** Combines multiple signals for comparison in a single scope
- **Constant Blocks:** Used for setting gain values
- **Signal Lines:** Used to create multiple parallel control paths

5.3 System Configuration

Three different configurations are implemented in the same model:

CASE 1: Without PID Control

- Direct connection of input to motor model
- Used as baseline for comparison

CASE 2: Manual PID Implementation

- PID implemented using Gain, Integrator, and Derivative blocks
- Allows observation of individual PID components

CASE 3: Tuned PID Controller

- Uses built-in PID block in Simulink
- Provides optimized system response

5.4 Output Visualization

All three outputs are combined using a Mux block and displayed on a single scope to allow direct comparison of system performance.

6. SIMULATION PARAMETERS

Parameter	Value
Transfer Function	$\frac{1}{0.5s + 1}$
Solver	ode45
Simulation Time	10 sec
Step Input	$0 \rightarrow 1$ at 5 sec

7. PID Parameters

Case	Kp	Ki	Kd
Manual PID	3	2	1
Tuned PID	5	3	1

8. SYSTEM PARAMETERS

8.1

Case 1: Without PID

The open-loop system response is governed by:

$$G(s) = \frac{1}{0.5s + 1}$$

Observations:

- Slow response
- Steady-state error present
- Poor tracking

8.2

Case 2: Manual PID Implementation

PID components are implemented using:

- Gain blocks
- Integrator

- Derivative block

Observations:

- Improved response
- Reduced steady-state error
- Moderate overshoot

8.3 Case 3: Tuned PID Controller

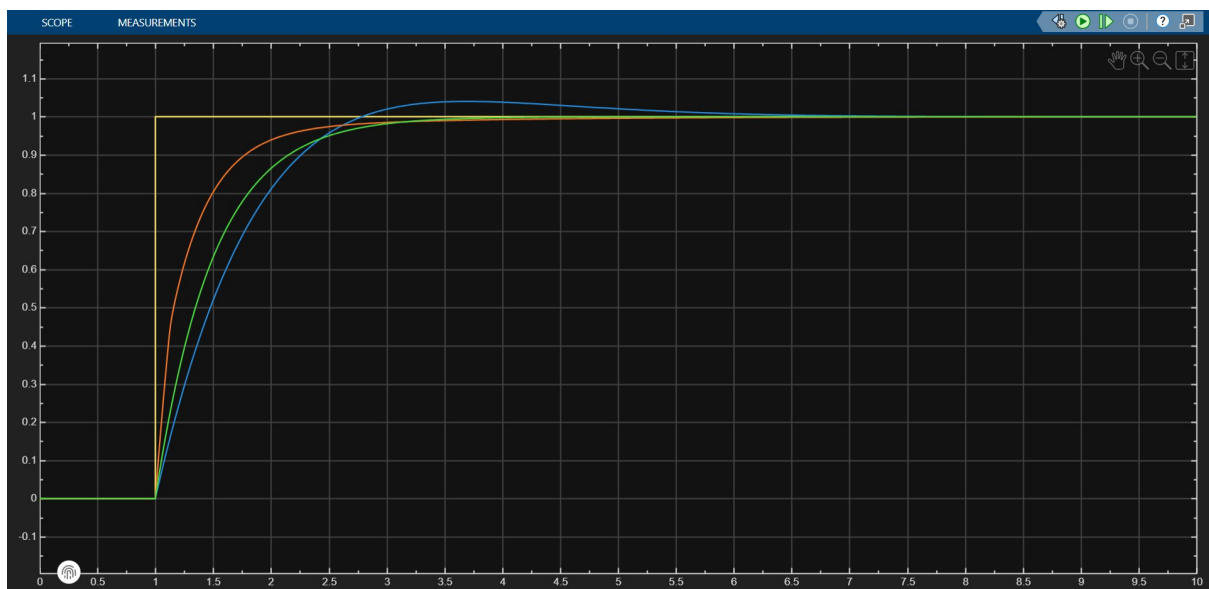
Using optimized gains:

$$K_p = 5, K_i = 3, K_d = 1$$

Observations:

- Fast rise time
- Minimal overshoot
- Quick settling
- Accurate tracking

9. RESULTS



Interpretation:

- The simulation results show three different system responses:

- Yellow line → Input
- Other curves →
- The **blue curve** represents the system without PID control, showing slow response and poor tracking.
- The **green curve** represents the manually tuned PID controller, showing improved performance.
- The **orange curve** represents the tuned PID controller, which provides the fastest response and best stability.

The tuned PID controller demonstrates superior performance compared to other configurations.

Conclusion from Parameter Variation

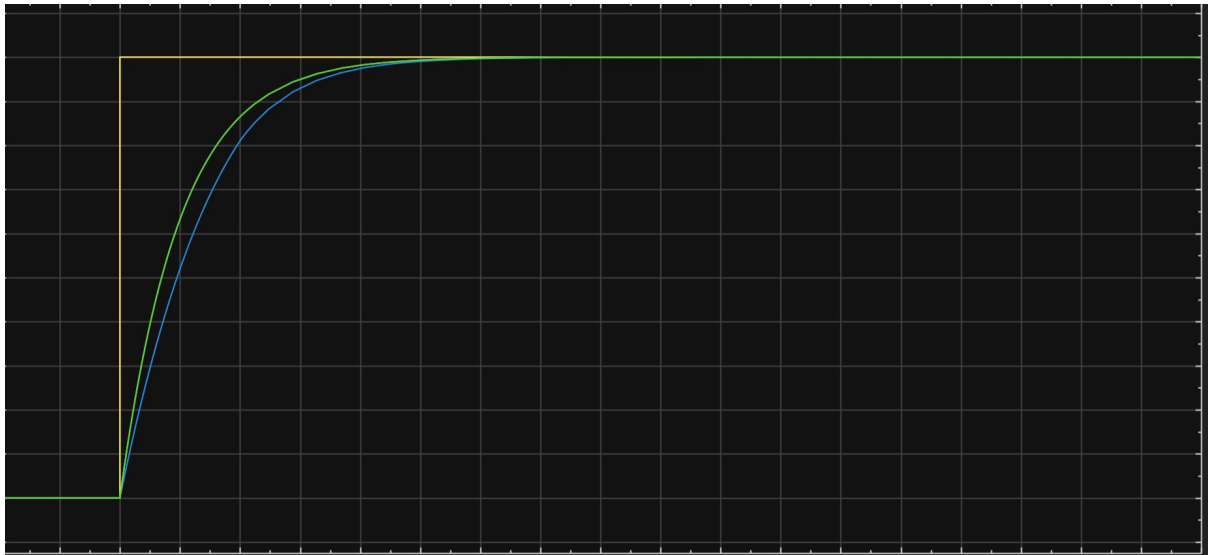
From simulation results, it is observed that:

- K_p mainly affects speed of response
- K_i removes steady-state error
- K_d improves stability and reduces oscillations

Proper tuning of these parameters is essential to achieve optimal system performance.

10. VARIATION OF PARAMETERS

10.1 Case 1: When step is varied from 1 – 2



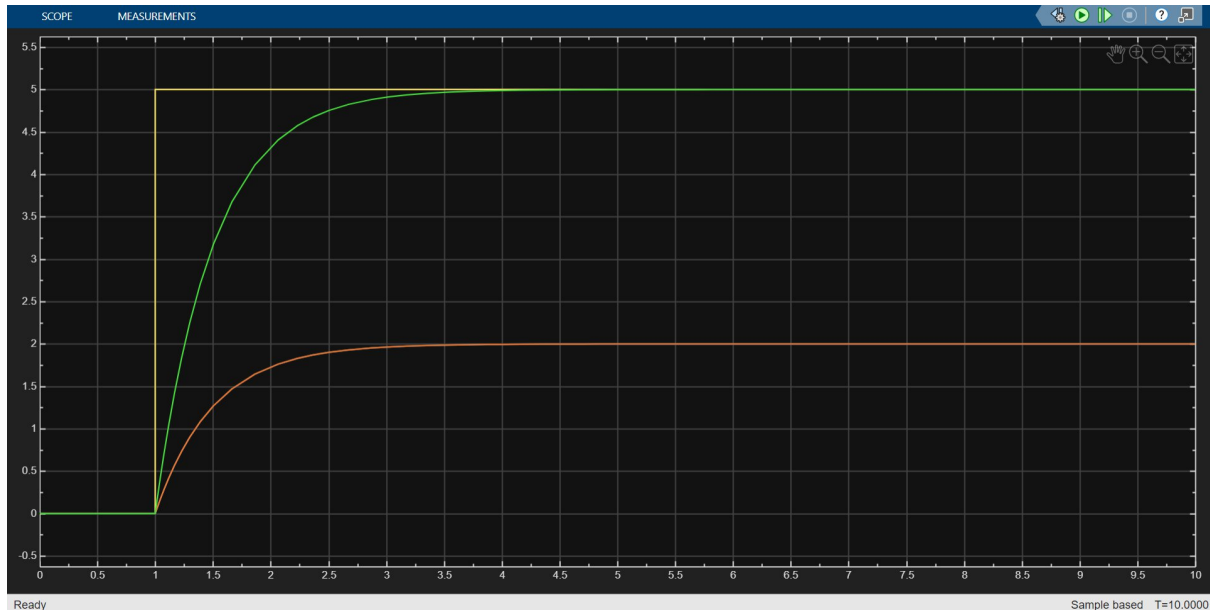
Observation:

The system shows a very rapid rise in output with significant overshoot. The response becomes more aggressive, and the settling time increases due to oscillations before stabilizing.

Reason:

The very high desired speed results in a large error signal. The PID controller generates a strong control action (high voltage), leading to rapid acceleration but also causing instability and higher overshoot before settling.

10.2 Case2: When step is varied from 2 - 5



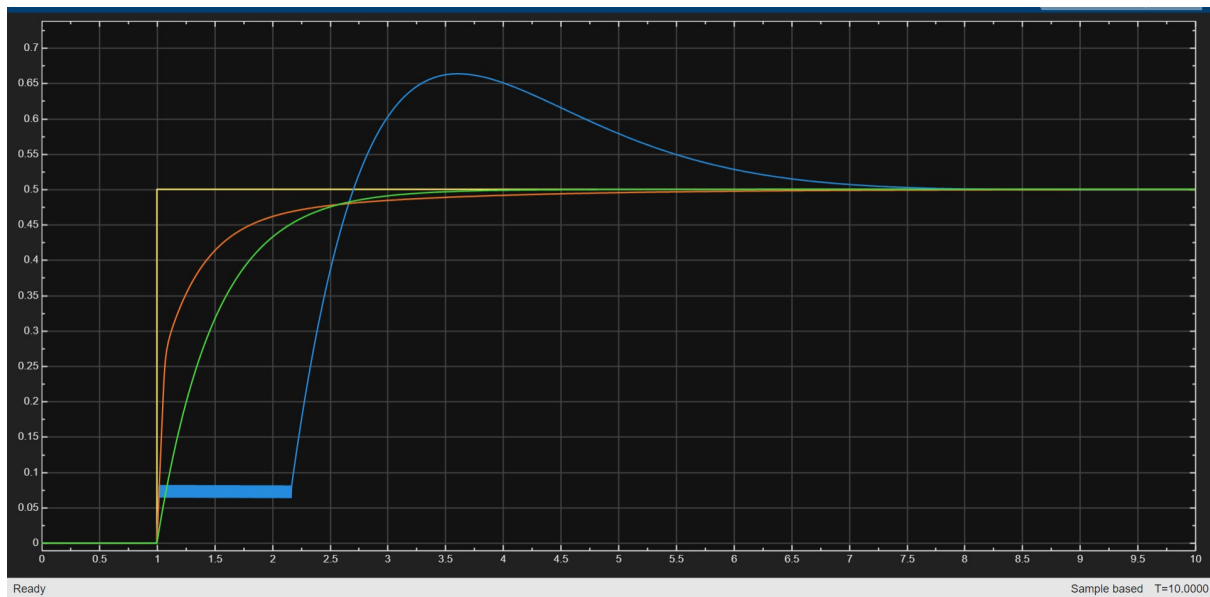
Observation:

The system reaches the desired speed with a balanced response. The rise time is faster compared to 0.5, and the system shows good stability with small overshoot.

Reason:

The error is higher than the previous case, so the PID controller applies a higher control voltage. This improves the response speed while maintaining stability due to proper tuning.

10.3 Case3: When step is varied from 5 - 0.5



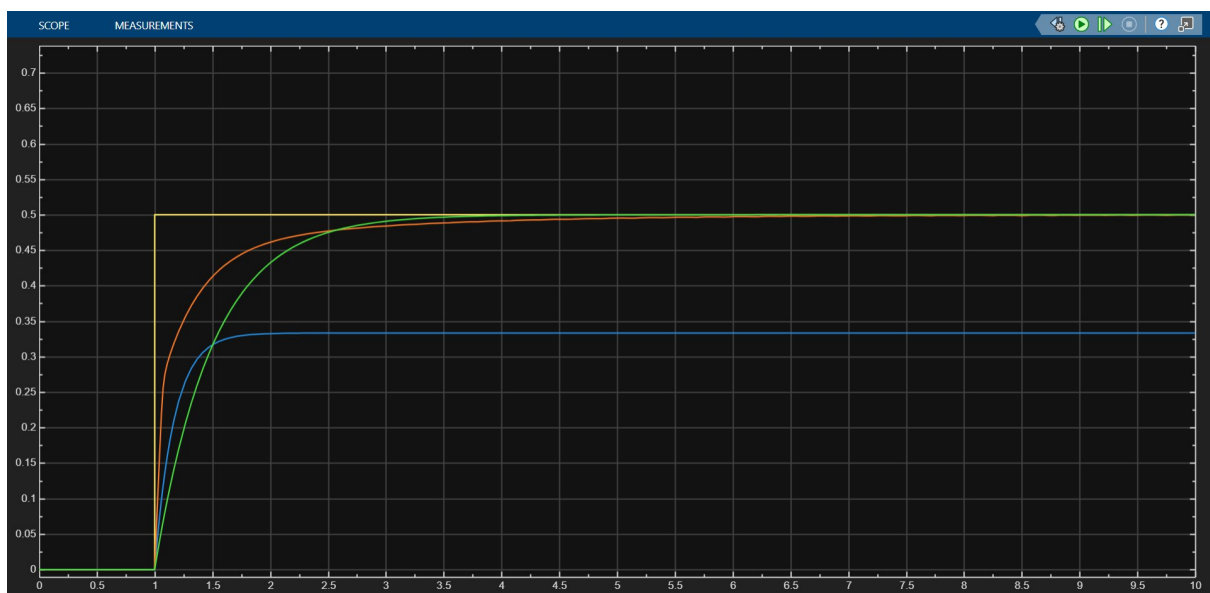
Observation:

The system reaches a lower steady-state speed with a smooth response. The rise time is moderate, and overshoot is minimal or negligible. The system stabilizes quickly.

Reason:

Since the desired speed is low, the initial error is small. The PID controller generates a smaller control signal (input voltage), resulting in a gradual increase in speed and reduced overshoot.

10.4 Case4: When $K_i=K_d=0$ and K_p is varied



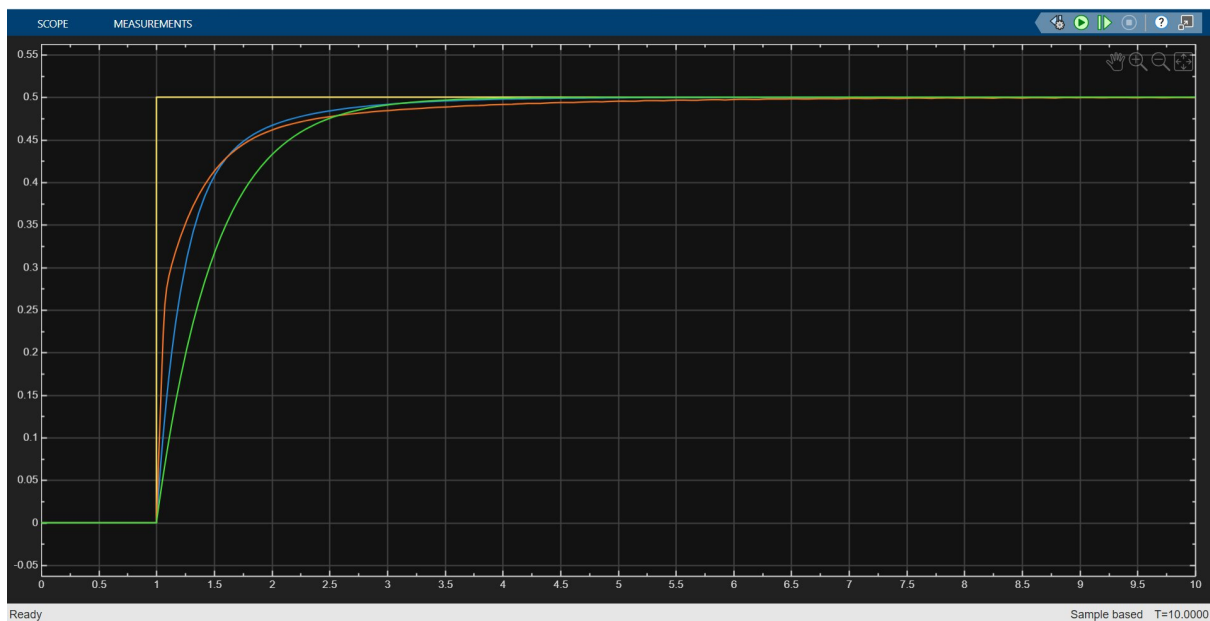
In this case, only proportional control was applied. It was observed that the system response improved in terms of speed compared to the open-loop system. However, the output did not reach the desired reference value, resulting in a steady-state error.

Observation:

- Faster response than open-loop system
- Presence of steady-state error
- No oscillations for moderate K_p values

Conclusion:

Proportional control improves response speed but cannot eliminate steady-state error.

10.5 Case5: When K_p constant , $K_d=0$ and K_i is varied

In this case, integral action was introduced while keeping K_p constant and $K_d = 0$. The value of K_i was varied to study its effect.

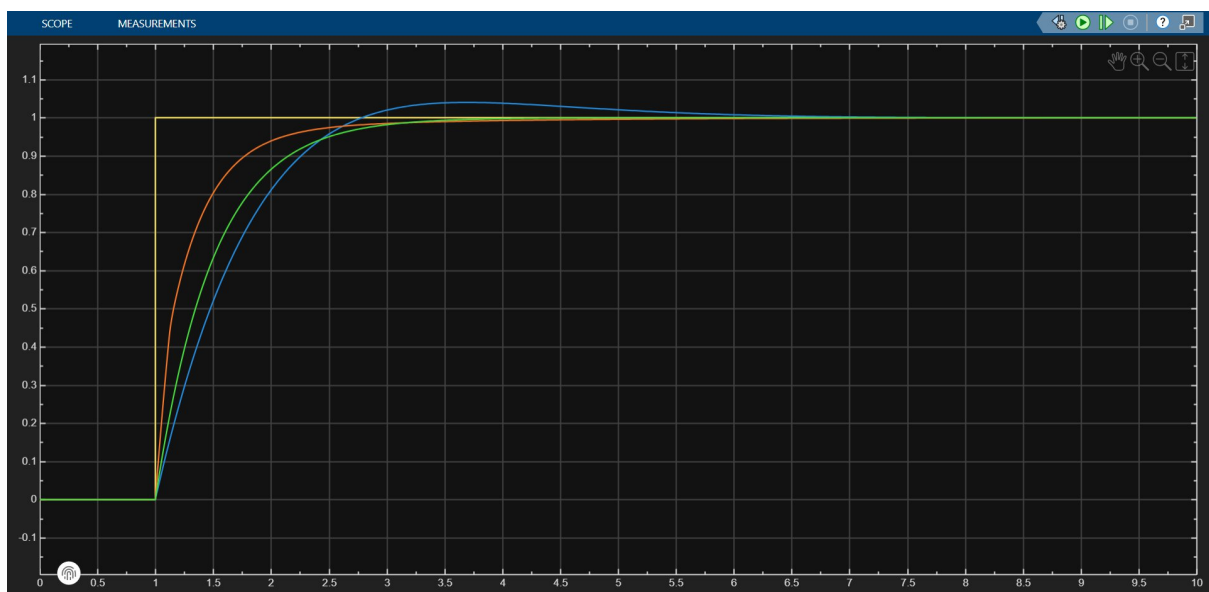
Observation:

- Steady-state error was eliminated as K_i increased
- System reached the desired reference value accurately
- Higher K_i values caused increased overshoot and oscillations
- Lower K_i values resulted in slower response

Conclusion:

Integral action is essential for eliminating steady-state error, but excessive integral gain can reduce system stability.

10.6 Case6: When $K_p=K_i=\text{constant}$ and K_d is varied



In this case, derivative action was added and K_d was varied while keeping K_p and K_i constant.

Observation:

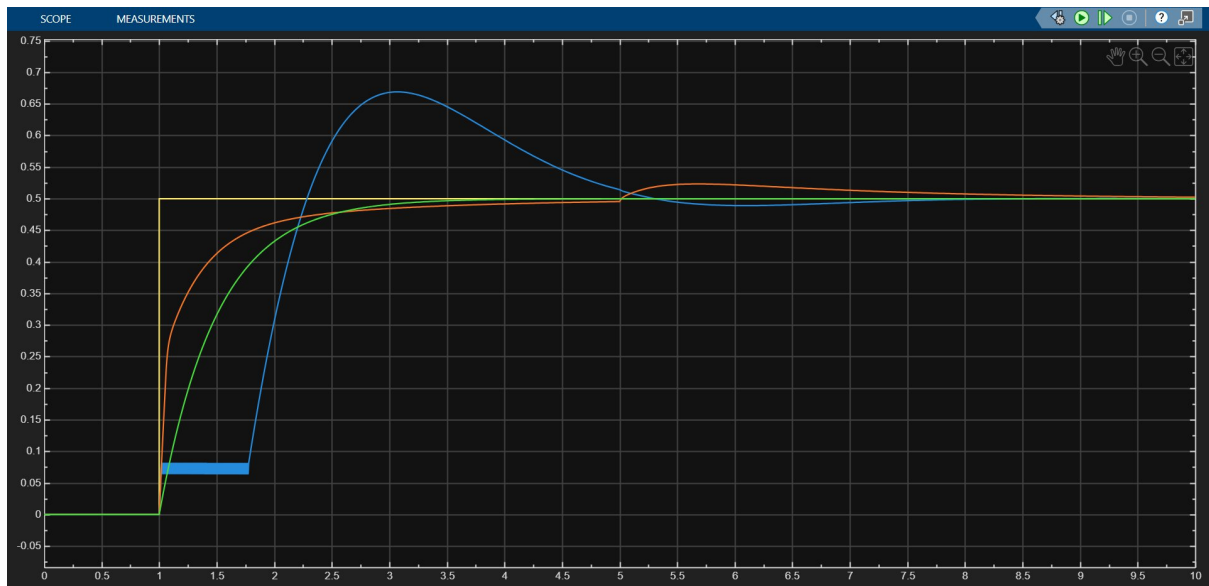
- Increasing K_d reduced overshoot
- System response became smoother and more stable
- Settling time improved due to better damping
- Very high K_d introduced noise and small fluctuations

Conclusion:

Derivative control improves system stability and reduces overshoot, but excessive derivative gain can make the system sensitive to noise.

The diagram illustrates a control system with two parallel paths for comparison:

- Without PID (Top Path):**
 - Input:** A step function $u(t)$ is fed into a summing junction.
 - Error Signal:** The output of the summing junction is $e(t)$.
 - Manual PID Controller:**
 - Proportional (Kp):** $e(t)$ is multiplied by a gain of 2.
 - Integral (Ki):** $e(t)$ is integrated ($\frac{1}{s}$) and then multiplied by a gain of 3.
 - Derivative (Kd):** $e(t)$ is differentiated ($\frac{\Delta u}{\Delta t}$) and then multiplied by a gain of 1.
 - Control Signal:** The outputs of the Kp, Ki, and Kd blocks are summed at a junction to produce $v(t)$.
 - Plant:** $v(t)$ is fed into a block labeled $\frac{1}{0.5s+1}$ (Plant).
 - Output:** The plant output $y(t)$ is fed back to the summing junction and also sent to an "Output" scope.
- Tuned PID (Bottom Path):**
 - Input:** A step function $u(t)$ is fed into a summing junction.
 - Error Signal:** The output of the summing junction is $e(t)$.
 - Tuned PID Controller:** $e(t)$ is fed into a block labeled $PID(s) \int$.
 - Control Signal:** The output of the PID block is summed with a disturbance $Input1$ (a step function) at another summing junction.
 - Plant:** The result is fed into a block labeled $\frac{1}{0.5s+1}$ (Plant 2).
 - Output:** The plant output $y(t)$ is fed back to the first summing junction and also sent to an "Output tf" scope.



To evaluate the robustness of the control system, a disturbance input was introduced in the tuned PID-controlled DC motor model. The disturbance was applied as a step signal at a specific time during system operation.

Observation:

- At the time of disturbance, the system output deviated from the desired reference value.
- The deviation was temporary, and the system response showed a corrective action.
- The PID controller acted to reduce the error caused by the disturbance.
- The output gradually returned to the reference value after a short duration.
- The system remained stable and did not exhibit sustained oscillations.

Reason:

The disturbance acts as an external input that affects the system output, creating a sudden error between the reference and actual output. The PID controller responds as follows:

- **Proportional (K_p):** Reacts immediately to the error and initiates correction.
- **Integral (K_i):** Accumulates the error over time and ensures that the steady-state error caused by the disturbance is eliminated.
- **Derivative (K_d):** Predicts the rate of change of error and helps in reducing oscillations, thereby improving system stability.

Due to the combined action of all three components, the controller compensates for the disturbance and restores the system to its desired operating point.

Conclusion:

The introduction of disturbance demonstrates the robustness of the PID-controlled system. While the disturbance temporarily affects the output, the controller effectively minimizes its impact and brings the system back to the desired reference value. This highlights the ability of PID control to maintain stability and accuracy even in the presence of external disturbances.

11. DISCUSSION

The proportional term improves system responsiveness, the integral term eliminates steady-state error, and the derivative term enhances stability by reducing oscillations. The combined PID controller effectively balances these characteristics, leading to optimal performance.

12. APPLICATIONS

- Motor speed control systems
- Robotics and automation
- Power electronics
- Embedded systems

13. CONCLUSION

The simulation results confirm that PID control significantly enhances the performance of the DC motor system. Among the three configurations studied, the tuned PID controller provides the best balance between speed, stability, and accuracy. This demonstrates the effectiveness of PID control in real-world engineering systems.

14. LEARNING OUTCOMES

- Understanding of dynamic system modelling
- Implementation of PID control
- Simulink-based system design
- Analysis of system performance

LIMITATIONS:

The model is based on a simplified linear approximation of a DC motor and does not consider nonlinear effects such as saturation and parameter variations.

FUTURE SCOPE:

The model can be extended to real-time hardware implementation and advanced tuning techniques such as auto-tuning or adaptive control.

